ACID Properties of a Transaction

The **ACID** properties are a set of guarantees that a database transaction must satisfy to ensure data integrity and reliability.[1] These properties ensure that data remains valid even in the event of errors, power failures, or other disruptions.[2] A transaction is a single logical unit of work that performs a series of operations, and the database management system (DBMS) ensures that all operations within it adhere to these properties.[3]

A: Atomicity

Atomicity ensures that a transaction is treated as a single, indivisible unit of work.[4] This means that either all operations within the transaction are completed successfully, or none of them are. If any part of the transaction fails, the entire transaction is rolled back to its initial state, and the database remains unchanged. There's no in-between state.

- **Example**: A money transfer from Account A to Account B. This transaction involves two operations: debiting Account A and crediting Account B. If the debit succeeds but the credit fails, the entire transaction is aborted, and the database state is restored to what it was before the debit. The money is not lost in a halfway state.

C: Consistency

Consistency ensures that a transaction brings the database from one valid state to another.[5] A valid state is one that adheres to all predefined rules, constraints, and triggers. The transaction itself must not violate any of the database's integrity rules.

- **Example**: In a bank database, a rule might state that the total balance of all accounts must be non-negative. A transaction that attempts to withdraw money from an account, resulting in a negative balance, would be rolled back because it violates the consistency rule.[6]

I: Isolation

Isolation ensures that concurrent transactions do not interfere with each other.[7] From a transaction's perspective, it appears as if it is the only transaction running on the database. The results of a transaction are not visible to other transactions until it is fully committed.[8]

- **Example**: Two people simultaneously try to buy the last remaining concert ticket. The database ensures that one person's transaction sees the ticket as available, processes the purchase, and then makes it unavailable. The second person's transaction sees the ticket as already sold and is either rolled back or given an error, preventing them from buying a ticket that's no longer there.

D: Durability

Durability ensures that once a transaction has been successfully completed and committed, its changes are permanently stored and will survive system failures, such as a power outage or crash.[9] The committed changes are written to non-volatile storage (like a hard drive), and the database system guarantees they will not be lost.

- **Example**: When a customer makes an online payment, and the transaction is committed, the bank guarantees that the payment record will not be lost, even if the system crashes a few moments later.